

Session 8 — Meet the app and plot the S&P 500

In class (90 min). Type along with the instructor. The project is **not graded**. The final exam is a closed-book MCQ: you must be able to **explain every line** you type.

Goal. Clone the stub, run Streamlit, and draw one live (or bundled) S&P 500 chart.

You leave with. Four tabs that open; Stocks shows a line for ^GSPC. Return tables wait until session 9.

Before you start

- Python 3.12 and `uv` installed.
- A GitHub account. The instructor gives you the **template repo URL** (Phase-0 stub).
- Work in the project folder (the one that contains `dashboard.py` and `pyproject.toml`).

Minimum path (sessions 8–10): Stocks + Bonds. **Stretch** (session 11): Commodities, then Currencies. Nobody is behind if they skip stretch.

Clone and install

In a terminal:

```
git clone <TEMPLATE-URL>
cd <folder>
uv sync
```

Windows: use PowerShell. If `uv` is not found, restart the terminal after installing `uv`.

Launch Streamlit

```
uv run streamlit run dashboard.py
```

A browser tab opens. Use the **sidebar** to switch Stocks, Bonds, Commodities, Currencies. Those last three are placeholders until later sessions.

Save any Python file: Streamlit **reruns the whole script**. That is normal.

How the app is wired

Open `dashboard.py`. You do **not** need to change it this week.

- `st.set_page_config` sets the title and wide layout.
- Each tab is an `st.Page` pointing at a file under `views/`.
- `st.navigation(...).run()` draws the sidebar and runs the selected page.

Open `views/equities.py`. You should see a `render()` function and a call to `render()` at the bottom. Streamlit executes that file when Stocks is selected.

Paths to bundled CSV files

Create `paths.py` next to `dashboard.py` if it is not already there:

```
from pathlib import Path

DASHBOARD_DIR = Path(__file__).resolve().parent
DATA_DIR = DASHBOARD_DIR / "data"
```

`DATA_DIR` must be the `data/` folder **next to this file**, not the terminal's working directory. That matters later on Streamlit Cloud.

Confirm `data/GSPC.csv` exists (bundled S&P 500 history). **Never overwrite files in data/ from Python**. Live downloads merge **in memory** only.

Shared loaders in `views/common.py`

Replace the stub with the helpers below (type with the instructor; do not paste blindly if you cannot explain a line).

```

"""Shared helpers for the Cross-Asset Market Monitor."""

from __future__ import annotations

from datetime import date, timedelta
from typing import Tuple

import pandas as pd
import streamlit as st
import yfinance as yf

from paths import DATA_DIR

DEFAULT_START_DATE = date(1927, 12, 30)
DEFAULT_END_DATE = date.today()

def _storage_filename(symbol: str) -> str:
    """Yahoo names like ^GSPC cannot be filenames."""
    return symbol.replace("^", "").replace("=", "_")

def _read_price_csv(path, symbol: str) -> pd.Series | None:
    if not path.exists():
        return None
    frame = pd.read_csv(path, index_col=0, parse_dates=True)
    frame.index = frame.index.tz_localize(None)
    column = "Close" if "Close" in frame.columns else frame.columns[0]
    return frame[column].rename(symbol)

def _load_bundled_series(symbol: str) -> pd.Series | None:
    return _read_price_csv(DATA_DIR / f"{_storage_filename(symbol)}.csv", symbol)

def _merge_live_and_stored(symbol: str, live: pd.Series | None) -> pd.Series:
    stored = _load_bundled_series(symbol)
    parts: list[pd.Series] = []
    if stored is not None:
        parts.append(stored)
    if live is not None and not live.empty:
        parts.append(live.dropna())
    if not parts:
        return pd.Series(dtype=float, name=symbol)
    combined = pd.concat(parts).sort_index()
    return combined[~combined.index.duplicated(keep="last")].rename(symbol)

def get_available_date_bounds(symbol: str) -> tuple[date, date] | None:
    bundled = _load_bundled_series(symbol)
    if bundled is None or bundled.empty:
        return None
    return bundled.index.min().date(), bundled.index.max().date()

def _live_fetch_start(
    symbols: Tuple[str, ...], requested_start: date, requested_end: date
) -> date | None:
    starts: list[date] = []
    for symbol in symbols:
        bounds = get_available_date_bounds(symbol)
        if bounds is None:
            starts.append(requested_start)
            continue
        stored_latest = bounds[1]
        if stored_latest >= requested_end:

```

```

        continue
    starts.append(max(requested_start, stored_latest))
if not starts:
    return None
return min(starts)

@st.cache_data(show_spinner=False, ttl=60 * 60)
def _fetch_live_prices(
    tickers: Tuple[str, ...],
    start_date: date,
    end_date: date,
) -> pd.DataFrame:
    if not tickers:
        return pd.DataFrame()
    end_exclusive = end_date + timedelta(days=1)
    try:
        price_frame = yf.download(
            list(tickers),
            start=start_date,
            end=end_exclusive,
            auto_adjust=True,
            progress=False,
        )
        close_prices = price_frame["Close"] if "Close" in price_frame else price_frame
        if isinstance(close_prices, pd.Series):
            close_prices = close_prices.to_frame(name=tickers[0])
        close_prices.index = close_prices.index.tz_localize(None)
        frames = []
        for symbol in tickers:
            if symbol not in close_prices.columns:
                continue
            series = close_prices[symbol].dropna()
            if not series.empty:
                frames.append(series.rename(symbol))
        if frames:
            return pd.concat(frames, axis=1).sort_index()
    except Exception:
        pass
    return pd.DataFrame()

def download_data(
    tickers: Tuple[str, ...],
    start_date: date,
    end_date: date,
    *,
    use_live: bool = True,
) -> pd.DataFrame:
    if not tickers:
        return pd.DataFrame()
    tickers = tuple(tickers)
    live_start = _live_fetch_start(tickers, start_date, end_date) if use_live else None
    live_frame = (
        _fetch_live_prices(tickers, live_start, end_date)
        if live_start is not None
        else pd.DataFrame()
    )
    merged = []
    for symbol in tickers:
        live_series = live_frame[symbol] if symbol in live_frame.columns else None
        series = _merge_live_and_stored(symbol, live_series)
        if not series.empty:
            merged.append(series)
    if not merged:
        return pd.DataFrame()

```

```

    fetched = pd.concat(merged, axis=1).sort_index()
    mask = (fetched.index >= pd.Timestamp(start_date)) & (
        fetched.index <= pd.Timestamp(end_date)
    )
    return fetched.loc[mask]

def preprocess(data: pd.DataFrame) -> pd.DataFrame:
    if data.empty:
        return data
    return data.ffill().dropna()

def chart_layout(**kwargs) -> dict:
    layout = dict(height=420, template="plotly_white", margin=dict(t=60, b=40))
    layout.update(kwargs)
    return layout

```

Note. `@st.cache_data` remembers the Yahoo result for one hour. The **second** run of the same download is faster. The exam may ask what happens on the second run.

Watch out. `yf.download` uses an **exclusive** end date, so we pass `end_date + one day`. Yahoo often fails from university Wi-Fi or later from Cloud — then `_merge_live_and_stored` still shows `GSPC.csv`.

Stocks tab: one line chart

In `views/equities.py`:

```

from datetime import date

import pandas as pd
import plotly.graph_objects as go
import streamlit as st

from views.common import (
    DEFAULT_END_DATE,
    chart_layout,
    download_data,
    preprocess,
)

SP500_TICKER = "^GSPC"

def render() -> None:
    st.header("Stocks")
    start = date(2020, 1, 1)
    end = DEFAULT_END_DATE
    prices = download_data((SP500_TICKER,), start, end, use_live=True)
    prices = preprocess(prices)
    if prices.empty:
        st.error("No S&P 500 data (Yahoo down and bundled CSV missing).")
        return
    series = prices[SP500_TICKER]
    fig = go.Figure()
    fig.add_trace(
        go.Scatter(x=series.index, y=series, mode="lines", name="S&P 500")
    )
    fig.update_layout(
        **chart_layout(title="S&P 500", yaxis_title="Index level", xaxis_title="Date")
    )
    st.plotly_chart(fig, use_container_width=True)

```

```
render()
```

Reload the browser. You should see a line. If Yahoo is blocked, the line stops at the last date in `GSPC.csv` — that still counts.

Commit

```
git add views/common.py views/equities.py paths.py
git commit -m "session 8: S&P chart"
```

Next week

Lookback presets, start/end dates, and trailing / calendar **return tables**. Do not start Bonds yet.